你打印的地址是假的：虚拟内存到底替你做了什么

嵌入式 Linux 手记　2026-10-26　公众号文章 · 原链接 mp.weixin.qq.com

先猜输出。这段程序 fork 出父子两个进程，都打印全局变量 `x` 的地址和它的值：

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int x = 42;

intmain(void) {
    pid_t pid = fork();
    if (pid == 0) {
        x = 100;
        printf("child:  %p -> %d\n", &x, x);
    } else {
        wait(NULL);
        printf("parent: %p -> %d\n", &x, x);
    }
    return0;
}
```

先猜父子打印的地址是否相同、值是否相同，再猜同一份程序单独跑两次，打印的地址会不会变。三个问题的答案分别是：地址相同、值不同（一个 100 一个 42）、两次运行的地址不一样。三个"不正常"全出自同一只手：虚拟内存。篇 15 结尾说过，Cortex-A 上 Linux 驱动拿到的地址是"坐标的坐标"；这一篇进那层坐标的加工厂，看 `0x7ffd3a2b1c94` 这类地址从 CPU 手里到内存条之前，到底被谁经手。

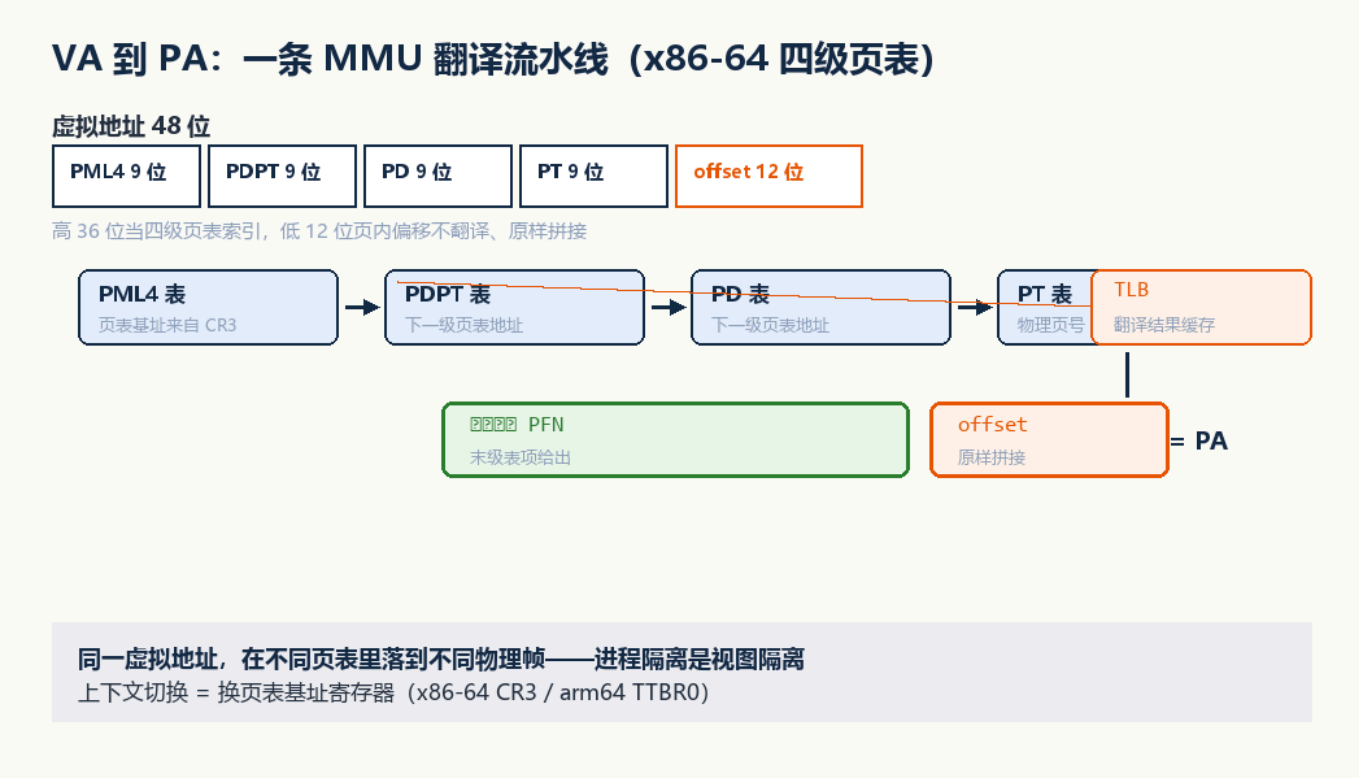
一、VA 到 PA：一条翻译流水线

先说结论：`%p` 打出来的地址是虚拟地址（VA, Virtual Address），不是物理地址（PA）。CPU 执行一条访存指令时，送进地址总线的并不是这个值，它要先过一道翻译。

流水线的起点是 CPU 里的 MMU（Memory Management Unit）。每条访存指令拿到 VA 后，MMU 把它切成两半：高位若干位当"虚拟页号"（VPN），低位若干位当"页内偏移"（offset，x86-64 与 arm64 常用 4KB 页，即低 12 位）。VPN 拿去查页表（page table）——那是内存里的一张映射表，逐级查下去，末端表项给出对应的物理页号（PFN）；PFN 与 offset 一拼，物理地址才算成形，这才被送上地址总线。

x86-64 的页表是四级结构：虚拟地址 48 位有效位切成 4 段各 9 位做四级索引，每级表项 8 字节，恰好一页 4KB 装 512 项，四级各取一页、末级拼出物理页号。arm64 同样多级（常规 48 位，扩展配置 52 位），中间级还有"块"描述符允许一次映射大块（L1 块对应 1GB、L2 块对应 2MB），末级只能映射 4KB 页。这套多级设计同时省了内存：没有进程会把 48 位地址空间用满，用不到的中间级页表根本不用存在。

查表不是没有代价的，所以 MMU 里还有一块缓存：TLB（Translation Lookaside Buffer）。刚查过的翻译结果缓存在 TLB 里，下一次访存命中即走，不必再翻四级表。TLB 的失效不由编译器和用户程序管理，只能在内核层显式处理：切换进程或修改页表后由内核刷新 TLB（x86 写 CR3、arm64 发 TLBI 指令），也是后面 `mmap` 、 `munmap` 语义的隐藏成本。

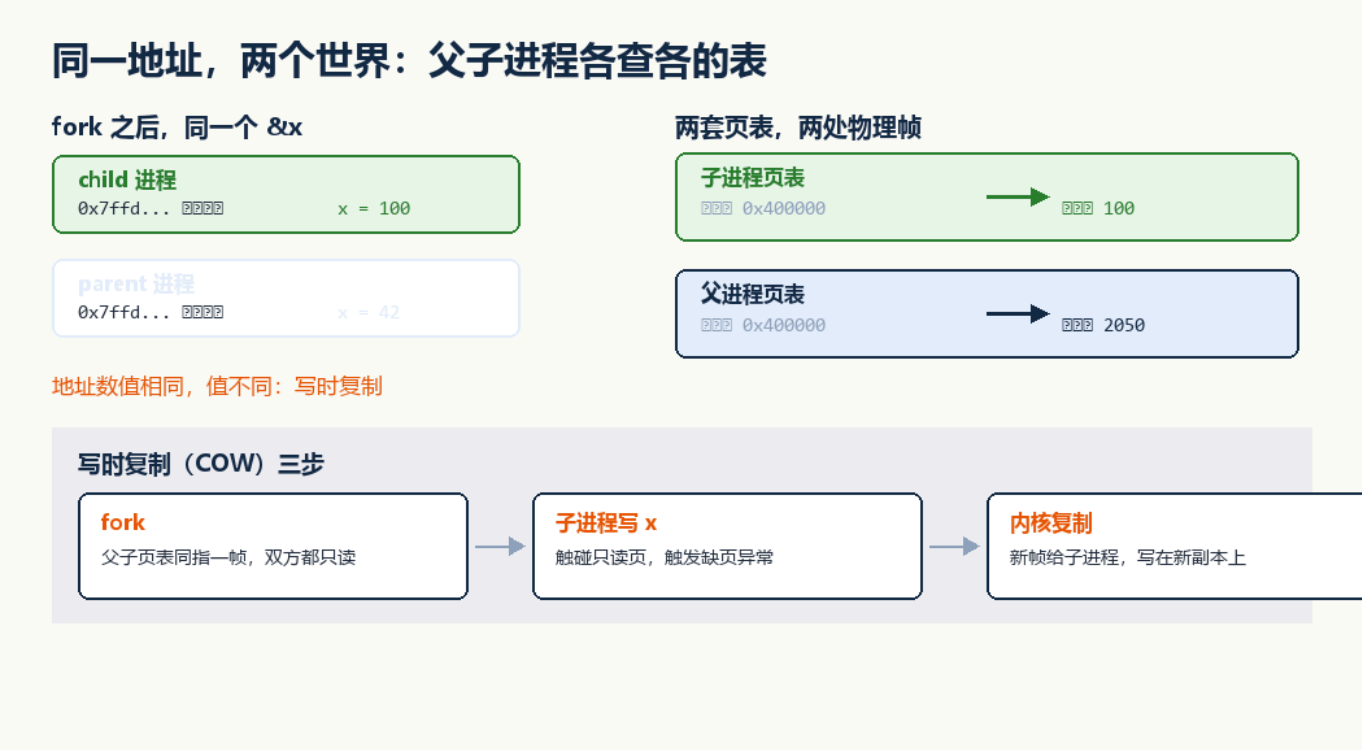


图：VA 到 PA 的翻译流水线。高 36 位逐级查表拿物理页号，低 12 位页内偏移原样拼接；TLB 缓存翻译结果。

二、每个进程一张表：隔离从哪来

页表不止一张，每个进程一张。同一个虚拟地址 `0x400000` ，在 A 进程的页表里指向物理帧 100，在 B 进程的页表里可以指向物理帧 2050——地址数值相同，落在两块完全不同的物理内存上。父子进程打印同一个 `&x` ，读到两个值，机制就在这： `fork` 给子进程复制了一张新页表，父子从此各走各的映射。

上下文切换的真相也随之显形：CPU 只有一套 MMU，"当前进程的页表"是寄存器里存的一个基址（x86-64 是 `CR3`，arm64 是 `TTBR0`）。进程切换时内核换掉这个寄存器指向的页表基址，下一瞬间同一个虚拟地址就指向了另一处物理内存。进程隔离不是硬件给每个进程发了独立的内存条，而是同一块内存条上，每张表各映射一份互不相干的视图。



图：同一地址，两个世界。父子进程的页表把同一个虚拟页映射到不同物理帧，子进程先写触发 COW 拿自己的副本。

这套视图还顺带解决了桌面系统最头疼的碎片与安全问题：每个进程都拥有从 0 开始排布的大片地址空间（代码段、数据段、堆、栈各就各位），分配器在连续虚拟地址上怎么排都行，物理页的碎片由内核在后台收拾；进程想摸别人的内存，页表里没有那条映射，摸到的只有异常。

三、这张表让出的三个魔法

页表一旦成为"地址的名义中枢"，许多桌面编程想当然的能力就浮出了水面。

魔法一，按需调页（demand paging）。程序启动或 `malloc` 大片内存时，内核并不急着分配物理页，只在页表里放一个"暂不存在"的标记，等程序第一次触碰该页，CPU 触发缺页异常，内核才补上一个填零的物理页。`malloc(1G)` 瞬间返回、只碰了几个字节的程序几乎不耗物理内存，"malloc 成功不等于内存到手"的机制层就在这。补的页为什么是零页？匿名映射的语义承

诺"新内存内容为 0"，这也把未初始化变量读成 0 的现实原因解释了一半（另一半是 .bss 不占文件，留给篇 23）。

魔法二，写时复制（COW）。`fork` 那一刻，内核不复制物理内存，只把父子两边的页表项都标成只读、指向同一批物理页。谁先写，谁触发缺页异常，内核给谁单独复制一份。开场的父子同地址不同值，正是子进程先写 `x`、触发 COW 拿了自己的副本。fork 一个上百进程的大程序从"复制整个地址空间"降为"复制页表加按需分页"，UNIX 的分裂能力有一半是这张表给的。

魔法三，`mmap`。把文件、设备或匿名内存接进页表：映射文件后按页读入、只读用过的部分；映射设备后用户态可以像访问数组一样访问硬件缓冲区；映射匿名内存就是 `malloc` 大块时的底层通道。文件 I/O 从"读进缓冲区"变成"地址空间里多了几页"，这条路径的细节留给系列后续篇目。

四、C 程序视角的三处印证

虚拟内存不只是操作系统课的术语，回看这个系列讲过的几个"为什么"，机制层全部收敛到这。

印证一，同一程序两次运行地址不一样。因为地址空间布局随机化（ASLR）：每次启动，内核把栈、堆、库的基址在随机偏移上排布，`0x7ffd3a2b1c94` 里的高位每次都不同。早年缓冲区溢出攻击靠猜固定地址下毒，ASLR 让地址本身成了秘密。

印证二，悬空指针崩与不崩的统一解释。篇 8 拆过：`free` 之后读旧值多数不崩，换个大块必崩。机制层一句话：小块 free 进分配器的空闲链，那块虚拟页的映射还在，读旧值不过是从一个"已易主"的合法页里取数，不崩；大块走 `mmap` 分配，`free` 即 `munmap`，页表项被摘掉，虚拟地址变成没人认领的野地址，再访问 MMU 查表失败，直接段错误。崩与不崩，是页表项在不在的问题，不是玄学。

印证三，空指针为什么几乎总是崩而不是"读到 0"。内核故意不映射地址 0 附近那一页，空指针解引用在页表里查不到任何有效映射，异常是最快的反馈。这个题目值得单独一篇，下一篇《段错误的根因：为什么解引用空指针通常会崩》专门拆，从信号、规范位一直拆到裸机边界。

写在最后

1. %p 打出的是虚拟地址：CPU 访存前由 MMU 查页表翻译成物理地址，VPN 查表、offset 原样拼接，4KB 页下低 12 位不参与翻译。

2. x86-64 四级页表、arm64 多级页表，用不到的中间级不必存在；TLB 缓存翻译结果，进程/页表变更后需刷新。
3. 每进程一张页表，同一虚拟地址在不同进程里映射到不同物理帧，进程隔离是"视图隔离"，不是内存条隔离；上下文切换的本质是换页表基址寄存器（CR3/TTBR0）。
4. 页表让出三个魔法：按需调页（malloc 大块不耗物理页）、写时复制（fork 不复制内存）、mmap（把一切接进地址空间）。
5. 悬空指针崩与不崩，是页表项在不在的问题（brk 小块的映射还在、mmap 大块已摘表）；ASLR 让地址本身成了秘密。

内存与底层系列才刚开头。下一篇回到那个让每个初学者都摔过一跤的问题：《段错误的根因：为什么解引用空指针通常会崩》，把 SIGSEGV、规范地址与裸机边界一次拆完。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。